Université de Poitiers
M2 IOI, M2 Informatique, M2 Mathématiques
2026–2027

Projet 2 : les réseaux de neurones

Modalités. Travail en binôme. À rendre : un notebook Python commenté (ou des scripts) et un court rapport présentant les résultats et leur analyse. Date de rendu : ...

1 Développement d'un perceptron

Conventions pour toute cette partie.
- Le coefficient d'apprentissage est fixé à $\alpha = 0{,}1$.
- Le seuil est traité comme un poids associé à une entrée constante égale à $+1$: pour une entrée $x = (x_1, x_2)$, on note $\tilde{x} = (1, x_1, x_2)$ et le potentiel du neurone vaut $v = w^T\tilde{x} = w_0 + w_1x_1 + w_2x_2$.
- Les vecteurs et matrices sont des tableaux NumPy, indexés à partir de 0.

1.1 Mise en place d'un perceptron simple

— *Créer une fonction*

```
y = perceptron_simple(x, w, active)
```

Cette fonction évalue la sortie d'un perceptron simple (un neurone) pour une entrée $x \in \mathbb{R}^2$.

Paramètres :
- `w` : poids synaptiques du neurone, vecteur de taille 3 ; `w[0]` correspond au seuil.
- `x` : entrée du neurone, vecteur de taille 2.
- `active` : fonction d'activation utilisée : $\varphi(v) = \text{sign}(v)$ si `active==0` ; $\varphi(v) = \tanh(v)$ si `active==1`.

Résultat :
- `y` : scalaire, sortie du neurone $y = \varphi(w^T\tilde{x})$.

Travail demandé :
— *Tester votre perceptron avec l'exemple du OU logique vu en cours, avec* $\varphi = \text{sign}$ $w_0 = -0.5$, $w_1 = 1$ *et* $w_2 = 1$. *Les entrées valent 0 ou 1, la sortie attendue vaut* -1 *pour* $(0, 0)$ *et* $+1$ *sinon.*
— *Afficher sur une même figure les quatre éléments de l'ensemble d'apprentissage et la droite séparatrice associée aux poids du neurone.*

La droite séparatrice est l'ensemble des points vérifiant $w_0 + w_1x_1 + w_2x_2 = 0$, soit $x_2 = -(w_0 + w_1x_1)/w_2$ (si $w_2 \neq 0$). Par exemple :

```
import numpy as np
import matplotlib.pyplot as plt

t = np.array([-0.5, 1.5])                # abscisses x1 du tracé
z = -(w[0] + w[1]*t) / w[2]              # ordonnées x2 correspondantes
plt.plot(t, z, 'k-')                     # droite séparatrice
plt.plot(X[0, yd == -1], X[1, yd == -1], 'o')   # classe -1
plt.plot(X[0, yd == 1],  X[1, yd == 1],  'x')   # classe +1
plt.xlim(t[0], t[1]); plt.ylim(-0.5, 1.5)
plt.show()
```

1.2 Étude de l'apprentissage

1.2.1 Programmation de l'apprentissage de Widrow-Hoff

— *Créer une fonction*

```
w, erreur = apprentissage_widrow(x, yd, epoch, batch_size)
```

Cette fonction retourne le vecteur de poids w obtenu par la règle d'apprentissage de Widrow-Hoff (descente du gradient). Pendant l'apprentissage, on utilise l'activation dérivable $\varphi = \tanh$: pour un individu $\tilde{x}_i$, la sortie vaut $y_i = \tanh(w^T \tilde{x}_i)$ et la correction associée est

$$\Delta w = \alpha \, (yd_i - y_i) \, (1 - y_i^2) \, \tilde{x}_i.$$

Paramètres :

- `x` : ensemble d'apprentissage, matrice de 2 lignes et N colonnes (un individu par colonne).
- `yd` : vecteur de taille N ; `yd[i]` vaut $+1$ ou -1 et indique la réponse désirée pour l'individu `x[:,i]` (classification à 2 classes).
- `epoch` : nombre maximal d'époques. Une époque correspond à un passage complet sur l'ensemble d'apprentissage.
- `batch_size` : nombre d'individus traités avant chaque mise à jour des poids ; les corrections Δw du lot sont additionnées puis appliquées. `batch_size=1` correspond à l'apprentissage en ligne (*online*), `batch_size=N` à l'apprentissage par lot complet.

Résultat :

- `w` : poids synaptiques du neurone après apprentissage, vecteur de taille 3 ; `w[0]` correspond au seuil.
- `erreur` : erreur quadratique cumulée sur chaque époque,

$$\text{erreur}[k] = \sum_{i=0}^{N-1} (yd_i - y_i)^2,$$

où y_i est la sortie (avec tanh) pendant l'époque k. C'est un vecteur dont la taille est égale au nombre d'époques réellement effectuées.

Les poids initiaux sont tirés aléatoirement (par exemple uniformément dans $[-0{,}5\,;\,0{,}5]$*). À la fin de chaque époque, afficher la droite séparatrice avec les points d'apprentissage, ainsi que le nombre d'individus mal classés (sortie* $\text{sign}(w^T \tilde{x}_i)$ *différente de* yd_i*). L'algorithme s'arrête dès que ce nombre est nul, ou au bout de* `epoch` *époques.*

1.2.2 Test 1 : données linéairement séparables

— *Charger les données du fichier* `p2_d1.txt` *:*

```
X = np.loadtxt('p2_d1.txt')      # matrice 2 x 50 : un individu par colonne
yd = np.concatenate((-np.ones(25), np.ones(25)))
```

L'ensemble d'apprentissage comprend 2 classes de 25 individus chacune en dimension 2. Les 25 premières colonnes appartiennent à la classe 1 (étiquette -1), les 25 suivantes à la classe 2 (étiquette $+1$).

— *Appliquer votre algorithme d'apprentissage. Afficher l'évolution de l'erreur et vérifier que la frontière obtenue sépare correctement les deux classes.*

— *Comparer l'apprentissage en ligne (*`batch_size=1`*) et par lot (*`batch_size=N`*).*

1.2.3 Test 2

— *Charger les données du fichier* `p2_d2.txt` *(même format et mêmes étiquettes que pour le test 1).*

— *Appliquer votre algorithme d'apprentissage de Widrow-Hoff. Comparer avec les résultats du test 1 : les classes sont-elles linéairement séparables ? L'erreur s'annule-t-elle ? Conclure sur les limites du perceptron simple.*

1.3 Perceptron multicouche

1.3.1 Mise en place d'un perceptron multicouche

— *Créer une fonction*

```
y = multiperceptron(x, w1, w2)
```

Cette fonction calcule la sortie d'un perceptron multicouche comportant deux neurones sur la couche cachée et un neurone sur la couche de sortie, pour une entrée $x \in \mathbb{R}^2$. La fonction d'activation de tous les neurones est la sigmoïde $\varphi(v) = \dfrac{1}{1+e^{-v}}$.

Paramètres :

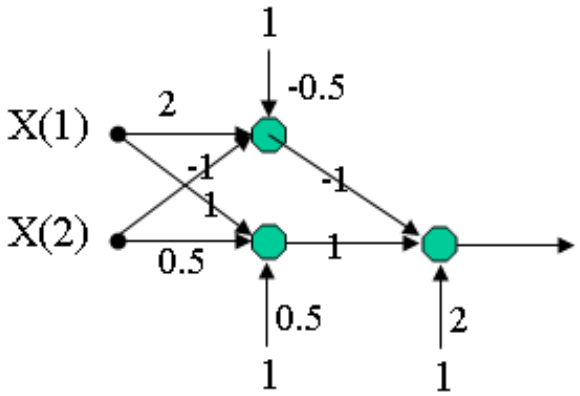
- `w1` : poids des 2 neurones de la couche cachée, matrice de 3 lignes et 2 colonnes. La colonne `w1[:,0]` correspond au premier neurone caché, la colonne `w1[:,1]` au second ; la ligne 0 contient les seuils.
- `w2` : poids du neurone de sortie, vecteur de taille 3 ; `w2[0]` correspond au seuil.
- `x` : entrée du réseau, vecteur de taille 2.

Résultat :

- `y` : scalaire, sortie du réseau.

Travail demandé :

— *Tester votre perceptron multicouche avec le réseau ci-dessous pour l'entrée $x = (1, 1)$.*



Les poids de ce réseau sont (les entrées constantes valent +1) :

$$\texttt{w1} = \begin{pmatrix} -0{,}5 & 0{,}5 \\ 2 & 1 \\ -1 & 0{,}5 \end{pmatrix}, \qquad \texttt{w2} = \begin{pmatrix} 2 \\ -1 \\ 1 \end{pmatrix}.$$

— *Comparer le résultat obtenu par programme avec la sortie calculée à la main.*

2 Réseaux denses et réseaux profonds : classification d'images

L'objectif de ce mini-projet est de réaliser un système de classification d'images réelles. La base utilisée est une base de test classique, la base Wang. Elle est composée de 1000 images (fichiers `0.jpg` à `999.jpg`) réparties en 10 classes de 100 images :

Numéros	Classe	Numéros	Classe
0–99	Afrique (scènes de vie)	500–599	Éléphants
100–199	Plages	600–699	Fleurs
200–299	Monuments	700–799	Chevaux
300–399	Bus	800–899	Montagnes
400–499	Dinosaures	900–999	Plats

La classe d'une image se déduit donc de son numéro : l'image `i.jpg` appartient à la classe $\lfloor i/100 \rfloor$. À l'intérieur de chaque classe, les images présentent une forte variabilité.

Le système à construire fonctionnera de la façon suivante :

— l'utilisateur propose au système une nouvelle image « inconnue » ;

— le système affiche la classe à laquelle il l'attribue.

Pour cela, il faut caractériser chaque image par des descripteurs. Vous allez développer deux approches : l'une reposant sur des descripteurs d'images calculés à l'avance, l'autre sur une stratégie profonde (*deep learning*) où les descripteurs sont appris par des couches de convolution.

Environnement. Les deux approches utilisent Keras/TensorFlow et scikit-learn.

2.1 Approche fondée sur des descripteurs

2.1.1 Descripteurs fournis

Le calcul des descripteurs n'est pas forcément complexe, mais il demande du temps de développement et des connaissances en traitement d'images. Nous vous fournissons donc directement les mesures calculées sur les images de la base Wang, pour 5 descripteurs :

— JCD (*Joint Composite Descriptor*),

— PHOG (*Pyramid Histogram of Oriented Gradients*),

— CEDD (*Color and Edge Directivity Descriptor*),

— FCTH (*Fuzzy Color and Texture Histogram*),

— *Fuzzy Color Histogram.*

Ces descripteurs mesurent des propriétés de couleur, de forme et de texture ; l'article fondateur de chacun d'eux est fourni.

Les mesures sont stockées dans le fichier `WangSignatures.xlsx`, qui contient un onglet par descripteur : `WangSignaturesJCD`, `WangSignaturesPHOG`, `WangSignaturesCEDD`, `WangSignaturesFCTH` et `WangSignaturesFuzzyColorHistogr`. Chaque onglet comporte 1000 lignes, une par image, et pas de ligne d'en-tête. La première colonne contient le nom de l'image ; les colonnes suivantes contiennent les valeurs du descripteur (leur nombre varie selon le descripteur).

Attention à l'ordre des lignes : les lignes sont triées par ordre alphabétique des noms de fichiers (`0.jpg`, `1.jpg`, `10.jpg`, `100.jpg`, `101.jpg`...) et non par numéro. Les étiquettes doivent donc être construites à partir de la première colonne, par exemple :

```
import pandas as pd
df = pd.read_excel('WangSignatures.xlsx', sheet_name='WangSignaturesJCD',
                   header=None, index_col=0)
X = df.values.astype(float)                          # mesures (1000 x d)
label = np.array([int(nom[:-4]) // 100 for nom in df.index])
```

— *Importer les 5 tableaux de mesures et construire le vecteur* `label` *donnant la classe (0 à 9) de chaque ligne.*

2.1.2 Classification par un perceptron multicouche

Le classifieur doit déterminer, à partir d'un ensemble d'apprentissage, la classe d'une image inconnue. Séparer aléatoirement la base en un ensemble d'apprentissage (80 %) et un ensemble de test (20 %), en conservant la proportion de chaque classe (`train_test_split(..., stratify=label)`). Les images de test jouent le rôle des images « inconnues ». Une partie de l'ensemble d'apprentissage peut être réservée à la validation pour régler les hyperparamètres.

Le classifieur sera un perceptron multicouche réalisé avec Keras/TensorFlow ; les hyperparamètres (nombre de couches, de neurones, fonctions d'activation, optimiseur, etc.) sont laissés libres.

- *Mettre en place un système qui, pour une image inconnue et son vecteur de mesures, propose une classe parmi les 10.*
- *Tester la classification avec chacun des 5 descripteurs séparément, puis avec leur concaténation. Analyser les résultats (matrice de confusion, taux d'erreur).*
- *Comparer différents choix d'hyperparamètres.*
- *Adapter votre architecture pour limiter le surapprentissage (overfitting) ou la disparition du gradient, si nécessaire.*
- *Comparer avec un classifieur des k plus proches voisins (KppV), dont vous optimiserez k et la distance par validation croisée.*

2.2 Approche profonde (apprentissage des descripteurs)

Nous proposons maintenant une stratégie entièrement fondée sur l'apprentissage : les descripteurs sont construits par des couches de convolution à partir des images elles-mêmes (dossier `Wang`). Les images de la base mesurent 384×256 ou 256×384 pixels : elles doivent être redimensionnées (ou recadrées) à une taille commune, par exemple 256×256, avant d'être données au réseau. Comme précédemment, l'étiquette se déduit du nom de fichier.

- *Mettre en place un système de classification qui, à partir d'une image, calcule des descripteurs par des couches de convolution.*

Par exemple, pour la partie extraction de caractéristiques :

```
from tensorflow import keras
from tensorflow.keras import layers

model = keras.Sequential()
model.add(keras.Input(shape=(256, 256, 3)))
model.add(layers.Conv2D(32, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D(pool_size=(2, 2)))
```

On ajoute ensuite des couches denses (*fully connected*) qui prennent la décision, c'est-à-dire qui proposent une classe parmi les 10 pour une image inconnue :

```
model.add(layers.Flatten())
model.add(layers.Dense(10, activation='softmax'))

model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',  # étiquettes entières 0..9
              metrics=['accuracy'])
```

- *Tester d'abord des architectures simples. Étudier l'influence des paramètres et l'évolution de la fonction de coût (apprentissage et validation).*
- *Comparer les résultats avec ceux des méthodes fondées sur les descripteurs.*
- *Tester des architectures plus complexes. L'augmentation de données (data augmentation) sera probablement nécessaire.*
- *Adapter votre architecture pour limiter le surapprentissage ou la disparition du gradient, si nécessaire.*
- *Tester une stratégie d'apprentissage par transfert (transfer learning), à partir d'un réseau pré-entraîné (par exemple VGG16 ou ResNet50 disponibles dans `keras.applications`).*